

TECHNISCHE UNIVERSITÄT
CHEMNITZ

**Extending Jupyter Notebook Reproducibility Analysis
Beyond GitHub**

A Cross-Platform Pipeline for Codeberg and Zenodo

Master Thesis - Draft

Submitted in Fulfilment of the Requirements for the Academic Degree
Master of Science

Department of Computer Science
Chair of Distributed and Self-organizing Systems

Submitted by: David Keci
Programme: M.Sc. Web Engineering
Supervisor: Prof. Dr. Sheeba Samuel
University: Chemnitz University of Technology
Draft date: 18 August 2026

Abstract

Jupyter notebooks combine executable code, narrative text, and generated results, but their apparent self-containment does not guarantee computational reproducibility. Existing large-scale reproducibility pipelines and the downstream FAIR Jupyter knowledge graph primarily operate on GitHub-hosted notebooks. This focus excludes resources distributed through alternative Git forges and research repositories whose access mechanisms and metadata models differ from GitHub. This thesis develops a cross-platform extension that processes GitHub, Codeberg, and Zenodo resources through a common reproducibility route while preserving the distinctions between Git repositories and archived research records.

The implementation introduces platform detection, platform-specific validation and acquisition, normalized metadata extraction, mixed-platform batch processing, and a copy-on-first-run database architecture. GitHub and Codeberg resources are validated and acquired with Git, whereas Zenodo records are validated through the Records API and downloaded as archives. Eight descriptive metadata fields are normalized and stored in a dedicated table. After acquisition, the existing platform-independent stages collect notebooks, infer requirements, create Python environments, execute notebooks, compare outputs, and record provenance and error information.

The database is subsequently exported into seven CSV datasets and transformed into RDF using YARRRML-derived RML mappings and Morph-KGC. An ontology extension represents platforms, Git repositories, archived records, notebooks, repository runs, notebook executions, reproducibility results, and notebook-classification activities. A preliminary build before the classification extension generated 222,048 unique triples and was checked with SPARQL competency queries. A hybrid classifier now compares weighted rules with a free local language model and requests human clarification on disagreement. The final comparative experiment and human-labelled classification evaluation remain future evaluation steps.

Keywords: computational reproducibility, Jupyter notebooks, metadata normalization, knowledge graphs, GitHub, Codeberg, Zenodo

Draft status: This document describes the implemented pipeline and the first knowledge-graph version. The final cross-platform experiment, final knowledge-graph review, and AI classification evaluation remain in progress.

Planned Table of Contents for an 80-Page Thesis

The following allocation defines the target length of the completed thesis. It is a writing plan rather than the current automatic page numbering; the final table of contents will update as the unfinished evaluation and discussion material is added.

No.	Section	Target pages	Length
—	Front matter	1–8	8
1	Introduction	9–14	6
2	Background and Related Work	15–24	10
3	Requirements and System Analysis	25–31	7
4	System Design	32–40	9
5	Pipeline Implementation	41–52	12
6	Knowledge-Graph Extension	53–61	9
7	Evaluation	62–70	9
8	Discussion	71–74	4
9	Conclusion and Future Work	75–77	3
—	Bibliography	78–80	3

Table 1.: Target page allocation for the completed 80-page thesis.

Appendices may follow page 80 and are not included in this target allocation. The largest remaining expansion belongs to the controlled cross-platform experiment, comparative result tables and plots, ontology review, and detailed discussion of threats to validity.

Contents

Abstract	i
Planned Table of Contents for an 80-Page Thesis	ii
List of Abbreviations	vii
1. Introduction	1
1.1. Motivation	1
1.2. Problem Statement	1
1.3. Research Goal and Questions	2
1.4. Contributions	3
1.5. Thesis Structure	3
2. Background and Related Work	4
2.1. Computational Reproducibility	4
2.2. The Three Source Platforms	4
2.3. Metadata Normalization	5
2.4. Environment Reconstruction and Notebook Execution	6
2.5. FAIR Data and Knowledge Graphs	6
2.6. Declarative Knowledge-Graph Construction	6
2.7. Research Gap	7
3. Requirements and System Analysis	8
3.1. Baseline and Scope	8
3.2. Functional Requirements	8
3.3. Non-Functional Requirements	8
3.4. Input and Output Contract	10
3.5. Data Model Requirements	10
3.6. Evaluation Requirements	11
4. System Design	12
4.1. Architecture Overview	12
4.2. Control Flow	12
4.3. Connector Strategy	13
4.4. Metadata Contract	13
4.5. Database Safety and Migration	14
4.6. Failure Model	14
4.7. Knowledge-Graph Design	14
4.8. Security and Robustness Considerations	15
5. Pipeline Implementation	16
5.1. Project Structure	16

5.2.	Repository Identity and Run Creation	16
5.3.	Single and Batch Modes	16
5.4.	Platform Detection and Validation	17
5.5.	Metadata Fetchers	17
5.6.	Metadata Storage	18
5.7.	Content Acquisition	18
5.8.	Notebook Registration and Discovery	19
5.9.	Requirements and Environment	19
5.10.	Execution, Error Analysis, and Comparison	19
5.11.	Hybrid Notebook-Purpose Classification	19
5.12.	Testing and Container Verification	20
6.	Knowledge-Graph Extension	21
6.1.	Relationship to FAIR Jupyter	21
6.2.	Relational Export	21
6.3.	Ontology Model	21
6.4.	YARRRML and RML Mappings	23
6.5.	Materialisation Process	23
6.6.	SPARQL Competency Questions	24
6.7.	Current Status	24
7.	Evaluation	25
7.1.	Evaluation Strategy	25
7.2.	Metadata Integration Test	25
7.3.	Database Inspection	25
7.4.	Knowledge-Graph Build Validation	26
7.5.	Planned Cross-Platform Experiment	26
7.6.	Hybrid Notebook Classification	27
7.7.	Reproducibility of the Evaluation	27
7.8.	Interim Assessment	28
8.	Discussion	29
8.1.	Answering the Research Questions	29
8.2.	Design Trade-offs	30
8.3.	Threats to Validity	30
8.4.	Operational and Ethical Considerations	31
8.5.	Integration Outlook	32
9.	Conclusion and Future Work	33
A.	Technical Appendix	34
A.1.	Main Pipeline Route	34
A.2.	Representative Database Queries	34
A.3.	Representative SPARQL Queries	35
A.4.	Build Commands	36
A.5.	Draft Completion Checklist	36

List of Figures

1.1. High-level scope of the cross-platform extension.	3
4.1. Layered architecture of the extended system.	12
6.1. Core classes and object properties in the extension ontology.	23

List of Tables

1.	Target page allocation for the completed 80-page thesis.	ii
2.1.	Platform differences relevant to the pipeline.	5
3.1.	Functional requirements.	9
4.1.	Connector dispatch strategy.	13
5.1.	Normalized metadata and source fields.	18
6.1.	Pipeline CSV exports and semantic purpose.	22
7.1.	Preliminary working database and graph counts.	26

List of Abbreviations

API	Application Programming Interface
CSV	Comma-Separated Values
DOAP	Description of a Project
FAIR	Findable, Accessible, Interoperable, and Reusable
IRI	Internationalized Resource Identifier
JSON	JavaScript Object Notation
KG	Knowledge Graph
NFDI	National Research Data Infrastructure
PROV-O	W3C Provenance Ontology
RDF	Resource Description Framework
RML	RDF Mapping Language
SPARQL	SPARQL Protocol and RDF Query Language
SQL	Structured Query Language
YARRRML	YAML-based human-readable RML representation

1. Introduction

1.1. Motivation

Jupyter notebooks have become a common format for communicating computational research. A notebook places source code, explanatory text, visualisations, and previously generated outputs in one document. This combination lowers the barrier to inspecting an analysis and can make the reasoning behind a result more transparent. However, the presence of code and output in the same file does not by itself make the result reproducible. A notebook can depend on an unspecified Python version, unavailable libraries, external files, hidden execution state, or a particular order of cell execution. When these conditions are not documented, a second researcher may be able to read the notebook but not recreate its results.

Large empirical studies demonstrate that this is not an exceptional problem. Pimentel et al. analysed approximately 1.4 million notebooks and reported recurring quality and reproducibility problems [10]. Samuel and Mietchen later studied biomedical notebooks connected to publications and showed that dependency installation, execution exceptions, and output differences all limit successful reproduction [13]. These studies provide both a motivation and a technical baseline for automated reproducibility analysis.

The available evidence is nevertheless strongly influenced by source selection. Existing data collection and execution pipelines commonly assume that notebooks are hosted in GitHub repositories. This assumption simplifies acquisition because a GitHub URL can be validated through Git and cloned into a local directory. It becomes problematic when research software is published elsewhere. Codeberg is also a Git forge, but exposes meta-data through a Forgejo-compatible interface rather than the GitHub API. Zenodo is more different: it is a research repository that publishes versioned records and downloadable files, and a record is not necessarily a Git repository. Treating all three platforms as interchangeable would therefore lose provenance and, in the Zenodo case, misrepresent the type of resource being processed.

The downstream representation has a similar limitation. FAIR Jupyter transforms a rich notebook reproducibility dataset into a knowledge graph that supports granular exploration [14]. Its original repository model is tied to GitHub-oriented data. A cross-platform pipeline requires a semantic model that can express the hosting platform, distinguish Git repositories from archived research records, and connect each resource to pipeline runs, notebook executions, and comparison results.

1.2. Problem Statement

The central problem addressed by this thesis is that a GitHub-specific reproducibility workflow cannot be extended to Codeberg and Zenodo by changing only a repository URL. The platforms differ at four levels:

1. **Validation.** Git repositories can be checked using Git remote operations, while

Zenodo records must be checked through an HTTP API.

2. **Acquisition.** GitHub and Codeberg resources can be cloned or pulled. Zenodo content must be selected from a record, downloaded, and potentially extracted.
3. **Metadata.** Equivalent concepts such as author, licence, keywords, and dates occur under different field names and structures. Some fields, especially a DOI, are native to Zenodo but normally absent from Git repositories.
4. **Semantics.** A Zenodo record is an archived research object and must not be asserted to be a Git repository merely because the downstream pipeline processes its files in a repository-like directory.

In addition, the original pipeline needs to preserve existing notebook-processing behaviour. Collection, environment reconstruction, execution, comparison, and error analysis are valuable and largely independent of the hosting platform after the files are locally available. Rewriting these stages for each platform would duplicate logic and make results harder to compare. The desired extension must therefore isolate platform-specific acquisition while converging into a shared execution route.

1.3. Research Goal and Questions

The goal is to design, implement, and evaluate a cross-platform extension of the computational reproducibility pipeline and its FAIR knowledge-graph layer. The system should support GitHub, Codeberg, and Zenodo with platform-appropriate connectors, normalize descriptive metadata into a common database schema, execute notebooks through the existing environment-reconstruction route, and publish the resulting provenance as linked RDF data.

The work is guided by the following questions:

- RQ1** How can one pipeline acquire and validate notebook resources from GitHub, Codeberg, and Zenodo without duplicating the platform-independent execution stages?
- RQ2** Which metadata fields can be normalized across the three platforms, and how can missing or structurally different values be handled consistently?
- RQ3** How can the FAIR Jupyter knowledge graph be extended to represent platform provenance, non-Git archived records, pipeline runs, executions, and reproducibility results?
- RQ4** Which technical checks and comparative metrics are required to evaluate the correctness and usefulness of the cross-platform extension?

The final comparative answer to RQ4 depends on a controlled experiment that is still in progress. This draft therefore distinguishes implemented technical validation from planned statistical evaluation.

1.4. Contributions

This thesis makes the following engineering and modelling contributions:

- a platform detector and connector architecture for GitHub, Codeberg, and Zenodo;
- mixed-platform batch processing that rebuilds the correct external URL from a stored platform and identifier;
- a copy-on-first-run database architecture that protects the source database and records all new results in a working copy;
- a normalized metadata schema and API adapters that produce the same eight fields for all supported platforms;
- a validation-first control flow that avoids saving metadata for unreachable resources;
- a Zenodo acquisition route that selects, downloads, and extracts record files instead of invoking Git;
- an ontology extension and seven mapping sets that connect repositories, archives, notebooks, runs, executions, comparison metrics, and classification activities;
- a hybrid notebook-purpose classifier that compares weighted rules with a free local language model and requests human clarification when their results differ;
- automated metadata and knowledge-graph tests plus an interactive demonstration of the database, pipeline, and semantic model.

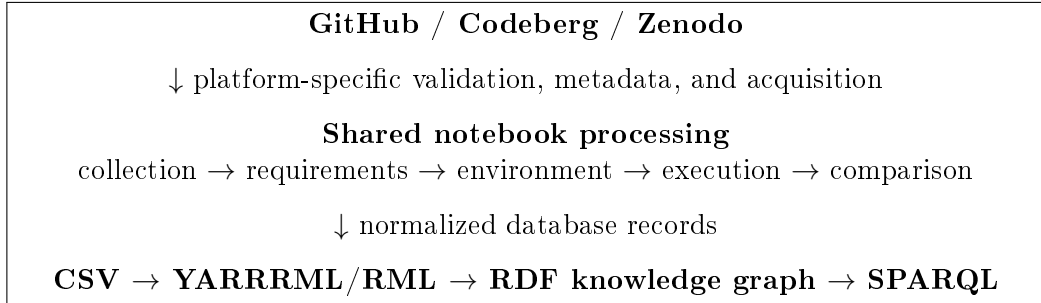


Figure 1.1.: High-level scope of the cross-platform extension.

1.5. Thesis Structure

Chapter 2 introduces computational reproducibility, platform characteristics, metadata normalization, and semantic publication. Chapter 3 derives requirements and defines the system boundary. Chapter 4 presents the architecture and data model. Chapter 5 explains the pipeline implementation, while Chapter 6 covers ontology and mapping extensions. Chapter 7 reports preliminary technical validation and specifies the remaining experiment. Chapter 8 discusses design trade-offs, threats to validity, and limitations. The thesis concludes with implemented results and future work.

2. Background and Related Work

2.1. Computational Reproducibility

Reproducibility terminology varies between disciplines, but the practical question in this work is concrete: can an automated system reconstruct a suitable software environment, execute a notebook, and obtain outputs that can be compared with the stored outputs? This question has at least three distinct outcomes. First, environment construction can fail before execution because a Python version or package cannot be installed. Second, the notebook can start but raise an exception. Third, the notebook can execute completely while producing output that differs from the original. Treating these outcomes as one Boolean value would hide useful evidence about the origin of a failure.

Notebooks complicate the problem because their interface permits non-linear interactive execution. A user can run cells out of order, redefine variables, or retain kernel state that is not reconstructible from the final document. Dependencies may be declared in `requirements.txt`, `setup.py`, environment files, or nowhere at all. Data can be loaded from relative files, remote URLs, or private services. Consequently, an execution system must combine explicit metadata with evidence obtained from the notebook content.

Pimentel et al. connected notebook quality indicators with reproducibility at large scale and highlighted execution order and software-engineering practices [10]. Samuel and Mietchen developed an automated biomedical pipeline that searched publication text for notebooks, located GitHub repositories, reconstructed environments, and compared rerun outputs [13]. Their published corpus provides the database baseline used by the present project. The work also demonstrates why error categories and output comparison must be retained rather than reporting only whether a command terminated successfully.

2.2. The Three Source Platforms

2.2.1. GitHub

GitHub is both a Git hosting service and an API provider. Repository content is acquired through Git, while descriptive information can be requested through the repository REST endpoint [5]. The response includes a repository name, description, owner, topics, licence information, and timestamps. Authentication is optional for public data but a token increases the practical request allowance. A GitHub-specific implementation can easily allow assumptions about host names, `/blob/` notebook URLs, and API response fields to spread into unrelated processing code. Removing these assumptions is a principal concern of the extension.

2.2.2. Codeberg

Codeberg is a community-run Git forge based on Forgejo. Content acquisition is therefore similar to GitHub: a remote can be checked with Git and cloned into a local directory. Metadata access is different. The repository endpoint follows the Forgejo API, and fields

such as the owner and licence can occur in more than one representation [3]. Codeberg notebook links use a `/src/` route where GitHub commonly uses `/blob/`. The similarity at the Git level and the difference at the API level make Codeberg a useful test of connector modularity.

2.2.3. Zenodo

Zenodo publishes research outputs as records with metadata and one or more files. Its REST API supports retrieving an individual record and downloading its files [2]. A record may contain a source archive, notebooks, data, documentation, or several unrelated files. It does not necessarily expose a Git remote. Validation must therefore confirm an API response, and acquisition must select a usable file URL. Zenodo also provides publication-oriented fields such as creators, keywords, licences, and persistent identifiers. The DOI is particularly important for linking a computational object to scholarly communication.

Table 2.1.: Platform differences relevant to the pipeline.

Aspect	GitHub	Codeberg	Zenodo
Resource type	Git repository	Git repository	Archived research record
Validation	<code>git ls-remote</code>	<code>git ls-remote</code>	Records API
Acquisition	clone or pull	clone or pull	file download and extraction
Metadata API	GitHub REST	Forgejo REST	Zenodo Records
Native DOI	normally absent	normally absent	commonly present
Stored identifier	owner/repository	owner/repository	numeric record ID

2.3. Metadata Normalization

Metadata normalization transforms semantically similar but structurally different source fields into a common representation. It is not the same as inventing missing values. The adapters in this work use an empty string when a source does not provide a value. This decision preserves comparability without claiming knowledge that the platform did not return.

Eight fields were chosen because they are available or meaningful across the sources: title, description, authors, licence, keywords, DOI, creation date, and update date. GitHub topics and Codeberg topics map to keywords. An owner or creator list maps to authors. A platform-specific licence object maps to one licence string. Zenodo’s DOI maps directly, whereas the field remains empty for ordinary Git repositories. The database additionally records the time at which metadata was fetched, allowing later analysis to distinguish source timestamps from pipeline provenance.

Normalization supports execution indirectly. Descriptive metadata does not replace dependency files, but it improves identification, licensing context, cross-platform comparison, and knowledge-graph publication. Environment metadata is handled separately by analysing requirements, setup files, runtime hints, and notebook imports.

2.4. Environment Reconstruction and Notebook Execution

The pipeline uses `pyenv` to select or install Python versions and create isolated environments. The tool’s version-selection mechanism allows multiple Python releases to coexist and supports project-specific selection [12]. Required packages are assembled from repository requirement files and imports detected in notebooks. This best-effort reconstruction cannot guarantee an identical historical environment, but it creates a repeatable automated procedure.

Notebook execution uses Jupyter `nbconvert`. In addition to converting notebooks into other formats, `nbconvert` can execute notebooks programmatically and save the resulting document [11]. The pipeline preserves an original/output pair so that a later comparison can distinguish identical, different, and nondeterministic cell results. Errors are classified from execution logs and stored with duration and cell-level context when available.

2.5. FAIR Data and Knowledge Graphs

The FAIR principles emphasise that digital research objects should be findable, accessible, interoperable, and reusable [15]. They do not require that every object be open, but they motivate persistent identification, rich metadata, standard representation, and machine-actionable relationships. RDF represents statements as subject-predicate-object triples and identifies resources with IRIs [4]. SPARQL provides a query language for matching and combining graph patterns [6].

FAIR Jupyter applies these ideas to a computational notebook reproducibility dataset. It converts relational exports into semantic triples and supports granular questions about publications, repositories, notebooks, and results [14]. The present work preserves that semantic-publication strategy but extends the source and provenance model. It must express three platforms and two resource categories while incorporating operational entities generated by the updated pipeline.

PROV-O provides general classes and properties for entities, activities, and agents [8]. It is therefore suitable for expressing a hosting platform as an agent, a repository or result as an entity, and pipeline runs and notebook executions as activities. Domain-specific classes can specialise these concepts while retaining compatibility with broader provenance tooling.

2.6. Declarative Knowledge-Graph Construction

Declarative mappings separate source-to-RDF transformation rules from procedural code. YARRRML provides a compact, human-readable representation for such rules [7]. These rules can be compiled to RML, which is then interpreted by a materialisation engine. Morph-KGC constructs RDF from heterogeneous sources and uses mapping partitions to improve scalability [1, 9].

This separation is valuable for the thesis for three reasons. First, reviewers can inspect how each CSV column becomes an RDF property without reading the database exporter. Second, the ontology and mapping can evolve independently of notebook execution. Third, per-table graph fragments can be validated and counted before they are combined into the final graph.

2.7. Research Gap

Existing execution research provides a strong GitHub-oriented reproducibility pipeline, while FAIR Jupyter provides semantic publication of its results. The missing element is an integrated cross-platform system that treats Git forges and research archives according to their actual access mechanisms and semantic types. The gap is not merely the lack of two download functions. It includes mixed-platform batching, safe identifier handling, normalized metadata, platform-aware database queries, archived-record modelling, operational provenance, and a repeatable path from the working database to RDF. These requirements motivate the design presented in the following chapters.

3. Requirements and System Analysis

3.1. Baseline and Scope

The implementation starts from an updated version of the computational reproducibility pipeline used for the biomedical notebook study. The baseline already contains the platform-independent stages for locating notebooks in a local repository directory, collecting dependencies, selecting Python versions, creating environments, executing notebooks, comparing outputs, and recording failures. The objective is not to replace these stages. The objective is to make the acquisition boundary explicit and to ensure that every supported source reaches the same downstream contract.

The downstream FAIR Jupyter project forms a second baseline. It already supplies an ontology centred on reproducibility, YARRRML and RML mappings for published dataset tables, and a Morph-KGC build route. The extension should remain compatible with that structure. No materialised graph is assumed to be source data; graph fragments and configurations are build outputs that can be regenerated.

The scope includes public GitHub repositories, public Codeberg repositories, and public Zenodo records that contain downloadable notebook material. Private repository authentication, Git submodule authentication, arbitrary archive formats, non-Python execution, and long-term graph hosting are outside the first implementation. Notebook-purpose classification is implemented as a first hybrid version that combines transparent rules with a free local language model. Its final labelled evaluation dataset remains part of the research evaluation.

3.2. Functional Requirements

Table 3.1 summarises the principal functional requirements.

The status distinction is deliberate. “First version” means that code exists and has produced a graph, but the final ontology review, mapping review, and experimental validation have not yet been signed off. This prevents a technical proof of construction from being confused with a complete research evaluation.

3.3. Non-Functional Requirements

3.3.1. Data Safety

The original database is treated as input evidence and must not be modified. The system uses a copy-on-first-run policy: if a working database is absent, the source is copied to `output/db/db.sqlite`; subsequent schema migration and execution results target only the working copy. If neither file exists, an empty working database can be created. This requirement was selected to make pipeline experiments recoverable and to separate source provenance from derived results.

Table 3.1.: Functional requirements.

ID	Requirement	Status
FR1	Detect GitHub, Codeberg, Zenodo, or an unsupported source from a repository URL.	Implemented
FR2	Process one user-supplied resource or a mixed-platform batch from SQLite.	Implemented
FR3	Validate Git resources through Git and Zenodo resources through the Records API.	Implemented
FR4	Normalize eight descriptive metadata values and store them without duplicate rows.	Implemented
FR5	Clone or update Git resources and download/extract Zenodo resources.	Implemented
FR6	Register supplied notebook paths or discover notebook files automatically.	Implemented
FR7	Reuse requirement inference, environment creation, execution, comparison, and error analysis.	Implemented
FR8	Export seven relational datasets and transform them into an ontology-conformant RDF graph.	First version
FR9	Validate graph structure with SPARQL competency queries.	First version
FR10	Classify notebooks and compare reproducibility by category.	First version

3.3.2. Comparability

All platforms must produce the same database fields and enter the same execution functions after acquisition. Platform-specific adapters may differ internally, but a normalized metadata array must always use the same field order. Run status names and result metrics must not depend on the platform. These constraints permit later grouping by platform without comparing incompatible measures.

3.3.3. Traceability

Every processing attempt must be represented separately from the persistent repository resource. A failed validation, failed environment, failed execution, and successful reproduction attempt require distinguishable statuses, messages, and durations. Logs must be written per resource. The knowledge graph must connect resources, runs, executions, and results instead of flattening them into repository attributes.

3.3.4. Regenerability

The RDF graph must be derived from versioned ontology and mapping files plus the working database. Temporary configurations, logs, split graphs, and the combined N-Triples file are outputs. A user must be able to delete outputs and reproduce them by rerunning the build command. The build must fail clearly if an expected mapping is missing or invalid.

3.3.5. Portability and Operability

The main pipeline is implemented as Bash modules because the baseline uses shell orchestration. External JSON is transformed with `jq`, relational state is managed with SQLite, and HTTP access uses `curl`. The KG builder uses Python because Morph-KGC and RDFLib provide the necessary graph construction and validation libraries. Container-based testing is retained for clean dependency verification, while repository-specific Python environments use `pyenv`.

3.4. Input and Output Contract

In single-resource mode, the required input is a repository or record URL. Notebook paths, setup paths, and requirement paths are optional semicolon-separated values. A supplied GitHub notebook URL is normalized by removing the host, owner, repository, `/blob/`, and branch prefix. A Codeberg `/src/` path is treated equivalently. A Zenodo path can be supplied after extraction, but normally notebooks are discovered automatically because the archive structure is not known before download.

In batch mode, each database row supplies a normalized identifier, platform, notebook paths, setup paths, and requirement paths. The system reconstructs an external URL according to the platform. The normalized repository field uses `owner/repository` for Git hosts and a bare numeric record ID for Zenodo. Platform-aware lookup prevents two hosts with the same path string from colliding.

The pipeline output consists of database rows, per-resource logs, executed notebooks, JSON comparison results, classification records, and summary messages. The KG output consists of seven CSV files, seven generated graph fragments, one combined N-Triples file, per-mapping logs and configuration, and a JSON build summary.

3.5. Data Model Requirements

The relational design separates stable resources from repeated activities. The core tables are:

- **repositories**: normalized identifier, platform, notebook/setup/requirement paths, and counts;
- **repository_metadata**: one normalized descriptive metadata row per repository;
- **notebooks**: notebook path and language linked to a repository;
- **repository_runs**: one complete attempt to process a resource;
- **notebook_executions**: execution status, timing, error information, and cell counts;
- **notebook_reproducibility_metrics**: original/executed comparison values.

Separating **repository_metadata** avoids overloading the operational repository row. The foreign key **repo_id** is unique, allowing an upsert to refresh metadata without creating duplicates. The platform remains on the repository row because it participates in identity, routing, and batch URL reconstruction.

3.6. Evaluation Requirements

Technical validation requires at least one public resource from each platform, a clean database copy, assertions about normalized metadata, and inspection of stored rows. KG validation requires parseable ontology and mappings, expected CSV row counts, successful per-mapping materialisation, a parseable combined graph, and SPARQL queries for platform and class constraints.

The final research evaluation must go further. It should sample several resources per platform under a documented selection policy, freeze the experiment date and software versions, and compare acquisition success, metadata completeness, environment success, execution success, error categories, and output reproducibility. Since GitHub is strongly represented in the inherited dataset while Codeberg and Zenodo examples are newly selected, conclusions must report sample sizes and avoid interpreting platform differences that are actually sampling differences.

4. System Design

4.1. Architecture Overview

The system uses a layered design with a narrow platform-specific boundary. Figure 4.1 shows the intended separation. The entry layer checks dependencies, loads configuration, creates output directories, prepares the working database, and asks the user for single or batch mode. The connector layer detects, validates, describes, and acquires an external resource. The processing layer operates only on a local directory and normalized notebook paths. The persistence layer records relational evidence. The semantic layer exports, maps, and materialises the graph.

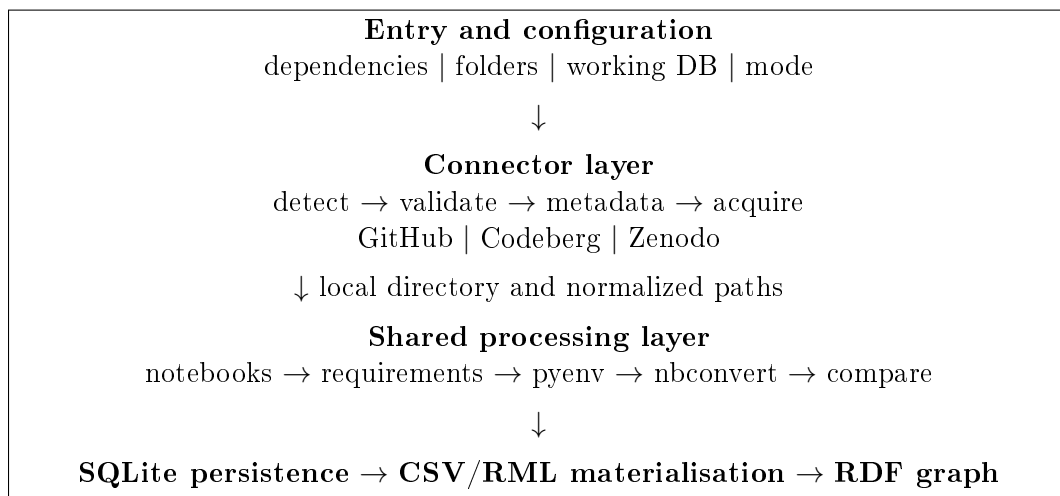


Figure 4.1.: Layered architecture of the extended system.

This architecture minimises change to proven functions. GitHub and Codeberg share Git validation and acquisition, but have separate metadata adapters. Zenodo has a separate validation and acquisition route. After local content is available, every source calls the same requirement, environment, execution, and comparison functions.

4.2. Control Flow

The main controller first resolves a repository ID. In single mode, it normalizes the input URL, detects the platform, and inserts a repository only if the normalized platform-identifier pair is absent. In batch mode, the ID already exists in the selected row. The controller then creates a `repository_runs` row with status `RUNNING`, records the start time, and enters the resource-processing function.

The order of validation and metadata collection is significant. An earlier draft collected metadata before checking reachability. That order could store a partial or misleading row when a resource was invalid. The final design validates first. Zenodo uses an API-oriented

validator, while GitHub and Codeberg use a Git-oriented validator. Only a valid source reaches the metadata controller.

The metadata controller is a dispatch function. It receives `repo_id`, `repo_url`, and `platform`; normalizes the external path; invokes one fetcher; checks that eight lines were returned; and passes them to a common save function. This explicit contract prevents platform fields from leaking into storage logic.

Content acquisition follows validation and metadata. A pre-existing local directory is updated with `git pull`; a new Git resource is cloned; a Zenodo record is downloaded and extracted. The directory must exist before notebook discovery. Supplied paths are registered before acquisition when known, while automatic discovery runs afterwards when paths are empty. This explains why notebook insertion can appear twice in the controller: the calls serve mutually exclusive input conditions and the insert function itself deduplicates rows.

The shared route then checks total and Python notebook counts, prepares requirements, creates an environment, executes notebooks, compares outputs, removes the temporary environment, and finalises the run. Every early return records a classified final status and elapsed time.

4.3. Connector Strategy

Table 4.1 maps platform decisions to connector behaviour.

Table 4.1.: Connector dispatch strategy.

Platform	Validator	Metadata fetcher	Acquisition
GitHub	<code>validate_repo</code>	<code>fetch_github_metadata</code>	clone/pull Git remote
Codeberg	<code>validate_repo</code>	<code>fetch_codeberg_metadata</code>	clone/pull Git remote
Zenodo	<code>validate_zenodo</code>	<code>fetch_zenodo_metadata</code>	API file download/extract

The dispatch table shows why “platform” and “resource type” are related but not identical. GitHub and Codeberg are different platforms with the same acquisition type. Zenodo is both a different platform and a different resource type. The ontology later represents this distinction with one platform class and two resource subclasses.

4.4. Metadata Contract

Each fetcher writes exactly eight newline-separated values in the following order:

1. title,
2. description,
3. authors,
4. licence,

5. keywords,
6. DOI,
7. creation date,
8. update date.

An array is used by the controller because shell functions cannot return structured objects directly. `mapfile` captures each emitted line as one array element. The controller verifies the array length before storage. Newline characters in Zenodo text are replaced with spaces so a multi-line description cannot be mistaken for additional fields.

Values are escaped immediately before SQL construction. The escape helper doubles single quotes in every text field. The save statement uses an upsert keyed by repository ID. The fetch timestamp is generated by SQLite, not accepted from an external API. This design separates source assertions from local provenance.

4.5. Database Safety and Migration

The configured paths distinguish `SOURCE_DB_FILE` from `DB_FILE`. On first execution, the source is copied to the working location. Schema preparation then creates missing pipeline tables and checks for added columns. Column-level migration is necessary because `CREATE TABLE IF NOT EXISTS` does not add columns to an existing table. A helper inspects `PRAGMA table_info` before an `ALTER TABLE` statement is executed.

The platform column defaults to `github` for compatibility with inherited rows. This is a migration convenience, not an assertion that an unknown new record is GitHub. New URLs are always detected explicitly. Batch rows with an empty legacy platform value are also interpreted as GitHub because the inherited dataset was GitHub-only.

4.6. Failure Model

Failures are first-class outputs. A finalised repository run records a status, message, and duration. Representative terminal states include invalid URL, invalid Zenodo record, missing repository directory, no notebooks, no Python notebooks, environment error, execution error, and success. Notebook-level execution rows carry more detailed error type, category, message, cell index, and count.

The failure model supports two kinds of analysis. Operationally, a user can find the stage that stopped a resource. Analytically, error categories can be grouped by platform or future notebook category. A generic shell exit code alone would not support either use case.

4.7. Knowledge-Graph Design

The semantic layer follows the relational activity structure instead of mapping every value to a repository literal. A `RepositoryResource` is hosted on a `RepositoryPlatform`, contains a `NotebookResource`, and has a `RepositoryRun`. A run has notebook executions, an execution identifies the notebook it used, and an execution has a reproducibility result. Inverse relations are defined where they improve navigation.

The design reuses PROV-O. Resources and results are entities, platforms are agents, and runs and executions are activities. `GitRepositoryResource` specialises both the common resource and DOAP's Git repository. `ArchivedResearchRecord` specialises the common resource and is disjoint with the Git class. This makes an invalid assertion about Zenodo detectable through ontology review or SPARQL tests.

4.8. Security and Robustness Considerations

API tokens are read from environment variables and are never written to the database. Public requests can run without tokens, but authenticated requests reduce rate-limit risk. URLs are still external input and should be treated cautiously. The pipeline restricts supported platform patterns and uses fixed API bases, reducing the chance that a stored host silently redirects metadata requests to an arbitrary service.

SQL escaping addresses quotes in fetched text, but parameterised database access would be safer and is recommended for a future Python-based storage layer. Archive extraction also deserves stronger controls: a production connector should reject absolute paths and parent-directory traversal inside downloaded archives. Finally, remote notebook execution is inherently risky. The present pipeline is intended for controlled research infrastructure; stronger process, network, and filesystem isolation is required before processing untrusted notebooks as a public service.

5. Pipeline Implementation

5.1. Project Structure

The executable pipeline is organised as shell modules under `src/`, an entry controller under `pipeline/`, shared configuration under `config/`, and a top-level `run.sh`. The separation follows responsibility rather than processing order. Database creation and migration are located in `src/db.sh`; validation in `src/checks.sh`; repository acquisition and metadata in `src/repo.sh`; notebook selection in `src/notebooks.sh`; requirement inference and environment management in their respective modules; and output comparison in the analysis module.

At startup, the entry script verifies required programs, sources the child modules, initializes configured directories, creates or copies the working database, and ensures all tables exist. Because sourced scripts run in the same shell process, exported variables such as `DB_FILE`, `REPOS_DIR`, `LOG_DIR`, `REPO_ID`, and `RUN_ID` are visible to later functions. This is different from executing a separate shell file as an unrelated process, which would not modify variables in its parent.

5.2. Repository Identity and Run Creation

Repository identity is based on a normalized identifier plus the platform. The normalizer removes the URL scheme and host, removes a Zenodo `record/` or `records/` prefix, and removes a trailing `.git`. GitHub and Codeberg therefore store values such as `octocat/Hello-World`; Zenodo stores a value such as `3362625`.

The `get_or_create_repo_id` function first checks whether the normalized identifier already exists. If present, it returns the existing ID. Otherwise, it inserts a repository row with the detected platform and supplied path information and returns `last_insert_rowid()`. Notebook insertion follows only after a repository ID is available, preserving the foreign-key relationship.

Every call to `process_repo` invokes `create_repository_run`. Unlike repository creation, run creation is intentionally not deduplicated. Two runs over the same repository represent two distinct attempts and may use different dates, source revisions, environments, or results. The function inserts a `RUNNING` row and exports its ID for execution records.

5.3. Single and Batch Modes

Single mode receives a URL from the user and creates the repository when necessary. Batch mode selects the next eligible database row that contains notebook information and has not already been represented in `repository_runs`. The number of resources is controlled by `TARGET_COUNT`. Processed IDs are retained during the loop so an exceptional row cannot be selected indefinitely.

Batch URLs are reconstructed with a platform switch:

Listing 5.1: Simplified mixed-platform URL reconstruction.

```
case "$PLATFORM" in
  github)  REPO_URL="https://github.com/${REPO_PATH}" ;;
  codeberg) REPO_URL="https://codeberg.org/${REPO_PATH}" ;;
  zenodo)  REPO_URL="https://zenodo.org/records/${REPO_PATH}" ;;
  *)       skip_unknown_platform ;;
esac
```

Carriage returns and quotes are removed from values read through SQLite's CSV mode. This is necessary when databases or fixtures have been prepared on Windows and are consumed inside a Linux container. Unknown platforms are logged and skipped rather than terminating the entire batch.

5.4. Platform Detection and Validation

The detector uses host patterns and returns one of `github`, `codeberg`, `zenodo`, or `unknown`. A single platform variable is computed near the beginning of `process_repo`; subsequent branches use that value rather than repeatedly parsing the URL.

GitHub and Codeberg call the existing Git validator. `git ls-remote` checks that the remote can be contacted and that it behaves as a Git repository without cloning its contents. Zenodo calls a dedicated validator because the remote URL is an HTML/API resource rather than a Git remote. The validator extracts the numeric record ID and performs a Records API request. Failure is recorded as `INVALID_ZENODO_RECORD`; Git failures are recorded as `INVALID_REPOSITORY_URL`.

Validation precedes metadata. This order guarantees that an unreachable source cannot create a new descriptive metadata row during the same run. It also provides a clearer log: a connector failure is attributed to validation rather than to a later JSON parser.

5.5. Metadata Fetchers

5.5.1. GitHub Adapter

The GitHub fetcher calls <https://api.github.com/repos/owner/repository>. When `GITHUB_TOKEN` is present, an Authorization header is added. `jq` transforms the JSON response into eight lines. The repository name becomes the title, the owner login becomes the author, the SPDX identifier becomes the licence, and topics are joined with semicolons. GitHub's creation and update timestamps are retained. DOI is emitted as an empty line.

5.5.2. Codeberg Adapter

The Codeberg fetcher calls the Forgejo repository endpoint at <https://codeberg.org/api/v1/repos/> using the normalized owner/repository path. Authentication is optional through the environment variable `CODEBERG_TOKEN`. The adapter is deliberately defensive. For authorship it prefers `owner.full_name`, then falls back to a login or username. The licence can be absent, an object, or another JSON value; the adapter selects an SPDX ID, key, name, or string representation in that order. Topics and timestamps are normalized like GitHub, and DOI remains empty.

5.5.3. Zenodo Adapter

The Zenodo fetcher accepts a full URL or normalized record ID. It calls <https://zenodo.org/api/records/id> and optionally uses `ZENODO_TOKEN`. The title and description come from the record metadata. Creator names are extracted from the creator array and joined with semicolons. Licence objects are reduced to an ID or title, keywords are joined, and the DOI is selected from the known record locations. Creation and modification timestamps use fallbacks because Zenodo response versions can expose publication and update dates differently. Embedded newlines are replaced with spaces before output.

Table 5.1.: Normalized metadata and source fields.

Normalized field	GitHub	Codeberg	Zenodo
Title	name	name	metadata.title
Authors	owner.login	owner full name/login	metadata.creators
Licence	license.spdx_id	licence object/value	licence ID/title
Keywords	topics	topics	metadata.keywords
DOI	empty	empty	doi / metadata DOI
Dates	created / updated	created / updated	created / published / modified

5.6. Metadata Storage

`fetch_and_save_repo_metadata` captures the selected adapter output with `mapfile`. If the number of values is not eight, the controller records a metadata message and does not call the database writer. When the contract is satisfied, `save_repo_metadata` escapes every text value and executes an upsert.

The new table is connected to `repositories` by `repo_id`. A unique constraint implements one current metadata record per repository. Reprocessing refreshes values and `fetches_at` rather than accumulating snapshots. Historical metadata versioning could be added later by replacing this constraint with a source timestamp and observation identity.

5.7. Content Acquisition

For Git repositories, the local directory is derived from the final URL component. A missing directory is populated with a shallow clone to reduce transfer size. An existing directory is updated with `git pull`. These operations are redirected into the per-repository log.

Zenodo uses three helpers. `zenodo_record_id` extracts the numeric identifier. `zenodo_download_url` inspects the API response and selects a ZIP file when present, otherwise the first downloadable file. `fetch_zenodo` creates the destination, downloads the selected object, and extracts a supported archive. The result is a local directory that conforms to the same downstream assumption as a Git clone.

This convergence point is central to the architecture. Notebook and environment modules do not call platform APIs and do not need to know whether files arrived through Git or an HTTP archive.

5.8. Notebook Registration and Discovery

`insert_notebooks_from_paths` splits a semicolon-separated argument, trims whitespace, normalizes complete notebook URLs, rejects non-`.ipynb` values, and inserts only missing rows. It stores the notebook path and a preliminary Python language value, not the entire notebook document.

If the user supplies no paths, discovery runs after acquisition with `find`. Absolute local paths are converted to paths relative to the resource directory. The resulting semicolon-separated value is logged and passed through the same insert function. Language statistics are requested only after supplied or discovered notebooks have been registered. This ordering fixed an earlier route in which an empty database check could stop before discovery had a chance to run.

5.9. Requirements and Environment

`process_requirements` combines explicit requirement files with imports found in selected notebooks. The goal is not to infer perfect dependency versions; it is to collect the best available installation set. Duplicate and irrelevant lines are removed before the final `requirements.txt` is written.

Python-version hints are searched in runtime, `pyenv`, and setup-related files. A fallback is used when no hint exists. `ensure_pyenv_version` checks installed versions and invokes installation when necessary. `setup_pyenv_env` creates a repository-specific environment, installs requirement packages, and processes supplied setup paths. Failure details are stored in shared error variables so the controller can finalise the run with a meaningful status.

The environment is temporary. `cleanup_pyenv_env` removes it after a failed setup, failed execution, or completed comparison. This reduces interference between resources and avoids selecting packages from a previous run.

5.10. Execution, Error Analysis, and Comparison

`run_in_pyenv_env` executes each notebook with Jupyter `nbconvert`. The executed notebook is written separately from the original. Execution rows capture status, duration, code-cell counts, and error information. When the command fails, `analyze_env_error` classifies the log, for example as a missing module or missing kernel, before the repository run is finalised.

Successful execution is not the end of reproducibility analysis. `compare_notebook_outputs` resolves repository and notebook IDs, locates original/output pairs, and calls a Python comparison program. The resulting JSON records identical, different, and non-deterministic cells and a reproducibility score. Database lookup now uses the shared working `DB_FILE` and includes platform-aware repository normalization.

5.11. Hybrid Notebook-Purpose Classification

The classification component first reads a notebook with `nbformat` and collects markdown, code, metadata, top-level Python imports, and output MIME types. Python imports are parsed with the standard-library `abstract-syntax-tree` module rather than by executing

notebook code. Compact excerpts are used for classification so image payloads and the full notebook document do not need to enter a model prompt.

Two independent methods use nine shared categories: data preparation, data analysis, visualisation, machine learning, simulation, tutorial, software development, mixed purpose, and uncertain. The rule-based method assigns weighted evidence from imports, code patterns, markdown terms, and output types. The second method calls a free local **gemma3:4b** model through Ollama. Ollama’s structured-output mode constrains the response to a category, secondary categories, confidence, reason, and evidence. The prompt treats notebook contents as untrusted data and uses temperature zero to reduce variation.

The reconciliation layer accepts a confident exact agreement as a provisional category. A disagreement, partial agreement, uncertain label, low confidence, disabled model, or unavailable local model sets **needs_human_review** and records a warning. A reviewer can assign the final category without overwriting either automated result. The database stores the notebook checksum, rule and model outputs, model and prompt versions, agreement status, warning, and human decision. Evaluation commands calculate coverage, accuracy, precision, recall, macro F1, per-category support, and the rule-model disagreement rate over reviewed rows.

5.12. Testing and Container Verification

The metadata test starts from a prepared working database, ensures migrations, loads the connector functions, and exercises one example per platform: GitHub’s public `octocat/Hello-World`, Codeberg’s `tplasdio/ipynb-py-convert`, and Zenodo record `3362625`. Assertions verify that each platform produces one metadata row and that expected non-empty fields are stored. The final query prints platform, repository, title, authors, licence, and DOI for inspection.

The test can be executed in a locally built Binder image. This verifies that the declared image contains Bash, Git, curl, jq, SQLite, Python, and the test files. Container testing does not replace per-repository pyenv environments; it validates the surrounding pipeline runtime.

A separate batch harness backs up the working database, swaps in a fixture, runs a limited batch, and restores the original through a shell trap. The restore step is important because the configured database path is fixed by the sourced configuration. Together, the tests cover connector normalization, schema migration, persistence, and mixed-platform orchestration while preserving the user’s current database.

6. Knowledge-Graph Extension

6.1. Relationship to FAIR Jupyter

The knowledge-graph component extends rather than replaces FAIR Jupyter. The upstream project provides the REPRODUCE-ME-based ontology, mappings for its published reproducibility dataset, and a Morph-KGC materialisation process. The cross-platform extension adds a separate ontology module and seven mappings for the working pipeline tables. Existing mapping files remain useful for publication and article-centred data; the pipeline mappings cover resources, operational results, and notebook-classification activities that were not represented by the original GitHub-oriented model.

This distinction matters when interpreting the final graph. It is a newly materialised graph produced from the local working database, but its vocabulary is designed as an extension of the FAIR Jupyter semantic layer. The build does not download a pre-existing materialised graph and append triples to it. Instead, it combines the extension ontology with newly generated fragments every time the builder runs.

6.2. Relational Export

Morph-KGC can consume several source types, but the existing FAIR Jupyter workflow uses CSV. A Python exporter therefore bridges SQLite and the mapping layer. Seven explicit SQL queries select and rename columns from repositories, metadata, notebooks, runs, executions, reproducibility metrics, and classifications. Explicit queries are preferable to `SELECT *` because they define a stable mapping contract and can add derived values such as external URLs, effective categories, and resource-type IRIs.

The exporter streams rows in batches rather than retaining a complete table in memory. Column names are taken from the cursor description and written as the header. A batch size suitable for SQLite reduces memory use while avoiding one write call per row. The output directory is created when necessary, and each exported count is printed for later comparison with the graph build summary.

6.3. Ontology Model

The ontology is identified by the <https://w3id.org/notebookfair> namespace and imports REPRODUCE-ME. It reuses PROV-O, DOAP, Dublin Core Terms, Schema.org, and RDF/RDFS concepts. Reuse reduces vocabulary duplication and makes the new assertions understandable to software that already recognises these standards.

6.3.1. Resource Classes

`RepositoryResource` is the common parent for a source-code repository or archived record processed by the pipeline. It is a subclass of `prov:Entity`. The neutral name is retained

Table 6.1.: Pipeline CSV exports and semantic purpose.

CSV file	Main RDF content
<code>repositories.csv</code>	resource type, label, identifier, URL, platform, notebook/setup/requirement counts
<code>repository_metadata.csv</code>	title, description, creator, licence, keywords, DOI, source dates, fetch time
<code>notebooks.csv</code>	notebook identity, language, and repository relationship
<code>repository_runs.csv</code>	processing activity, status, message, timing, and repository relationship
<code>notebook_executions.csv</code>	execution activity, notebook/run links, duration, errors, and cell counts
<code>notebook_reproducibility_metrics.csv</code>	result entity, execution relationship, comparison counts, and score

for compatibility with the relational table, but its definition is intentionally broader than a Git repository.

`GitRepositoryResource` is a subclass of both `RepositoryResource` and `doap:GitRepository`. GitHub and Codeberg rows use this type. `ArchivedResearchRecord` is also a subclass of `RepositoryResource`, but is declared disjoint with the Git class. Zenodo rows use the archived type. The disjointness statement documents a central conceptual correction: the ability to process extracted files does not make the original record a Git remote.

`RepositoryPlatform` is a subclass of `prov:Agent`. The ontology defines named individuals for GitHub, Codeberg, and Zenodo. A resource links to one of these individuals through `hostedOnPlatform`, which specialises `prov:wasAttributedTo`.

6.3.2. Notebook and Activity Classes

`NotebookResource` specialises the REPRODUCE-ME notebook class. It is connected to its parent resource through `containsNotebook` and the inverse `isNotebookOf`. `RepositoryRun` and `NotebookExecution` are PROV activities. A resource `hasRun`; a run `hasExecution`; and an execution records the `executedNotebook`. These relations permit a resource with several attempts and a run with several notebook executions without duplicating the resource.

`ReproducibilityResult` is a PROV entity generated by an execution. The forward property is `hasReproducibilityResult`; its inverse `resultOfExecution` specialises `prov:wasGeneratedBy`. Comparison counts and score are attached to this result instead of the repository, because multiple executions may produce different results.

6.3.3. Data Properties

Resource properties include a repository identifier and counts for notebooks, setup files, and requirement files. Platform individuals have a platform name. Notebook resources have a language. Run properties capture status and duration. Execution properties capture execution status, duration, total/executed code cells, error type, error category, message, cell index, and error count. Result properties capture identical, different, and nondeterministic cell counts plus a decimal reproducibility score.

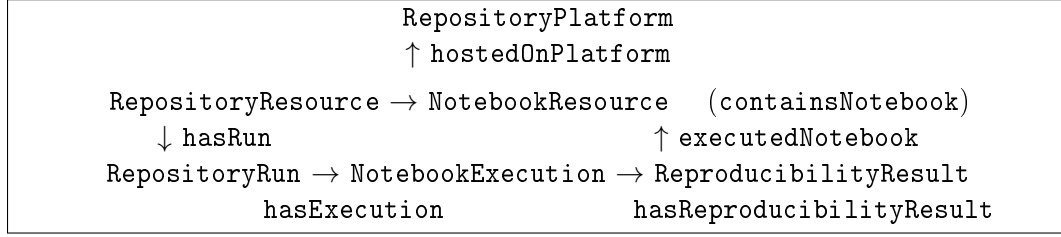


Figure 6.1.: Core classes and object properties in the extension ontology.

Descriptive metadata uses Dublin Core where possible: title, description, creator, licence, subject, identifier, created, and modified. The extension adds `metadataFetchedAt` because the retrieval time describes the local pipeline observation rather than the source resource.

6.4. YARRRML and RML Mappings

Each CSV has a dedicated YARRRML source file. Repository mappings create platform individuals and repository resources, choose the class IRI already calculated by the exporter, and connect the resource to its platform. Metadata mappings reuse the repository subject IRI, so descriptive properties enrich the existing resource rather than creating a disconnected metadata node.

Notebook mappings create notebook IRIs based on database IDs and emit both notebook-to-repository and repository-to-notebook links. Run mappings follow the same bidirectional strategy. Execution mappings connect executions to runs and notebooks. Metric mappings create result IRIs and link them to executions.

Stable IRI templates are essential for joining graphs generated from separate files. The templates use database IDs under paths such as:

Listing 6.1: Representative IRI templates.

```

https://w3id.org/notebookfair/repository/{id}
https://w3id.org/notebookfair/notebook/{id}
https://w3id.org/notebookfair/run/{id}
https://w3id.org/notebookfair/execution/{id}
https://w3id.org/notebookfair/result/{id}
https://w3id.org/notebookfair/platform/{platform}

```

The readable YARRRML rules are compiled into RML Turtle files under a pipeline mapping directory. The builder validates that exactly seven compiled mappings are present. This check prevents a silently incomplete graph when a file has not been compiled or copied.

6.5. Materialisation Process

`build_pipeline_kg.py` orchestrates the current build. It resolves paths relative to the FAIR Jupyter directory, checks that Morph-KGC is installed, parses the ontology with `RDFLib`, and parses every RML mapping as Turtle. Unless `-skip-export` is supplied, it invokes the CSV exporter against the working database.

The output area contains configuration, logs, and split-graph directories. Old files matching the generated patterns are removed so current counts cannot be confused with stale fragments. For each mapping, the builder writes a Morph-KGC configuration that points to one RML file and one N-Triples output. It measures execution time, preserves standard output and error in the mapping log, and parses the generated graph to count triples.

After all mappings succeed, RDFLib loads the extension ontology and each graph fragment into one graph and serialises `notebookfair-pipeline.nt`. A JSON summary records build time, source database, ontology path, combined graph path, total unique triples, CSV row counts, and per-mapping counts and durations.

The current build flow is:

SQLite → seven CSV files → YARRRML → RML → Morph-KGC
→ seven N-Triples fragments → combined graph → SPARQL tests.

6.6. SPARQL Competency Questions

Competency queries express the questions the graph must answer. They include:

- list resources and their hosting platforms;
- distinguish Git repository resources from archived research records;
- find notebooks contained in a selected resource;
- trace a repository run to its executions and results;
- group run or execution status by platform;
- retrieve DOI and descriptive metadata for Zenodo resources;
- detect any Zenodo resource incorrectly typed as a Git repository;
- obtain reproducibility scores and cell-difference counts.

A zero-result query can be a successful constraint test, for example when it asks for resources that are simultaneously Zenodo records and Git repository resources. Other queries require non-empty results and known platform counts. The tests should therefore define expected semantics per query instead of treating every empty result as a failure.

6.7. Current Status

The ontology, exporter, seven YARRRML mappings, compiled RML mappings, builder, and initial SPARQL tests have produced a complete first graph version. The seventh mapping represents notebook-classification activities, both automated categories, agreement status, model provenance, warnings, and human decisions. The graph is not considered final in this draft. Remaining work includes reviewing domains and ranges, checking all inverse relations, confirming literal datatypes, aligning the extension more closely with upstream FAIR Jupyter naming, expanding SPARQL assertions, and rebuilding after the final experiments. This status is reported explicitly so that a successful materialisation is not mistaken for a completed semantic evaluation.

7. Evaluation

7.1. Evaluation Strategy

The evaluation is divided into technical validation and comparative research evaluation. Technical validation asks whether the implemented connectors, schema, persistence, and graph builder behave according to their contracts. Comparative evaluation asks whether meaningful differences in metadata quality and reproducibility outcomes can be observed across platforms. The first part has preliminary results; the second remains in progress.

This separation avoids two common errors. First, passing a unit or integration test does not demonstrate that the system generalises to diverse research repositories. Second, a platform comparison based on one convenient example per host would not support statistical conclusions. The examples used for connector testing are therefore reported as fixtures, not as a representative sample.

7.2. Metadata Integration Test

The metadata test uses one publicly reachable resource for each connector:

- GitHub: <https://github.com/octocat/Hello-World>;
- Codeberg: <https://codeberg.org/tplasio/ipynb-py-convert>;
- Zenodo: <https://zenodo.org/records/3362625>.

The test prepares the working schema, creates or finds repository rows, invokes the metadata controller with the expected platform, and queries the resulting table. It asserts that each repository has exactly one metadata row and that the normalized title is non-empty. The Zenodo row additionally provides a DOI, testing a field that is expected to remain empty for ordinary Git repositories.

The test was executed inside a locally built container image. The output reported successful migrations for the platform, keywords, and DOI additions and ended with **[TEST] Metadata extraction and storage PASSED**. The printed inspection rows contained titles and authors for all three sources and DOI `10.5281/zenodo.3362625` for the Zenodo fixture.

The test demonstrates connector reachability, JSON adaptation, eight-field normalization, SQL storage, and cross-platform querying. It does not demonstrate completeness for all API response variants or rate-limit behaviour. Those concerns require a larger fixture set and error-response mocking.

7.3. Database Inspection

The inherited working database and current pipeline outputs provide the preliminary counts shown in Table 7.1. These numbers describe the current development database, not a balanced cross-platform experiment.

Table 7.1.: Preliminary working database and graph counts.

Item	Count
Repository resources exported	5,244
Notebook resources exported	27,274
Repository runs	116
Notebook executions	444
Reproducibility metric rows	444
Normalized metadata test rows	3
Unique triples in first combined graph	222,048

Most inherited resources are GitHub records from the baseline study. Consequently, the large repository and notebook counts must not be presented as evidence that Codeberg and Zenodo have been evaluated at the same scale. The platform column and new examples make mixed representation possible, but a controlled sample is still needed.

7.4. Knowledge-Graph Build Validation

The first full build performs several checks before reporting success. RDFLib parses the OWL extension, and each compiled RML file is parsed as Turtle. The exporter now writes seven CSV files and prints row counts. Morph-KGC generates one graph fragment per mapping, and every fragment is parsed again to count its triples. Finally, the combined graph is parsed and serialised as N-Triples.

The build produced 222,048 unique triples. The number is lower than the sum of all intermediate triple counts when two mappings produce an identical relationship, because RDF graphs are sets and duplicate triples collapse during combination. A JSON summary preserves the mapping-level counts and elapsed times for reproducibility.

SPARQL tests check that GitHub and Codeberg resources are typed as Git repository resources, Zenodo is typed as an archived record, resources connect to platform individuals, and operational entities are reachable through the expected property chain. A negative constraint query checks that no Zenodo record is simultaneously typed as a Git repository resource. Further tests of datatypes and cardinality remain part of the final KG review.

7.5. Planned Cross-Platform Experiment

The final experiment will use several resources from each platform. The selection procedure must be documented before results are collected. A practical design is a stratified sample with equal target counts per platform, limited to public resources that contain at least one Python notebook. Where possible, the sample should cover projects with and without requirement files, single and multiple notebooks, and different publication ages.

The experiment will record the following stages separately:

1. validation success;
2. content acquisition success;
3. metadata completeness;

4. notebook discovery success;
5. environment creation success;
6. notebook execution success;
7. output comparison category;
8. total and stage-specific duration.

Metadata completeness can be calculated as the fraction of eight normalized fields that are non-empty, both overall and per field. DOI should be analysed separately because its absence on Git platforms is expected and should not automatically imply lower metadata quality. Environment and execution failures should be grouped by the stored error categories. Reproducibility results should report the proportion of complete executions with identical, different, or nondeterministic cells.

7.6. Hybrid Notebook Classification

Notebook classification supports analysis rather than determining whether a notebook is allowed to run. The implemented first version uses data preparation, data analysis, visualisation, machine learning, simulation, tutorial, software development, mixed-purpose, and uncertain categories. It collects notebook markdown, imported packages, code tokens, metadata, and output types without executing code. Transparent weighted rules and a free local language model then produce independent categories and confidence values.

The methods are reconciled conservatively. Confident exact agreement produces a provisional category. Disagreement, partial agreement, uncertainty, low confidence, or model unavailability produces a warning requiring human clarification. The raw outputs remain stored after review. Automated tests cover collection, weighted evidence, structured model responses, agreement, disagreement, SQLite persistence, and human correction. A real local integration fixture containing NumPy and Matplotlib was classified as visualisation by both methods. This fixture demonstrates operation, not classification accuracy.

Final evaluation still requires a manually labelled sample and documented annotation guide. The implementation can export the review queue and report method coverage, accuracy, precision, recall, macro F1, per-category support, and disagreement rate. A random subset of agreed results must also be reviewed because agreement does not guarantee correctness. Only after classification quality is measured should category be compared with environment and execution outcomes.

7.7. Reproducibility of the Evaluation

The experiment should record the pipeline commit, container image identifier, operating system, Python/pyenv versions, run date, and API access mode. The input resource list should contain persistent identifiers or exact repository revisions. Logs and database outputs should be retained in a versioned result package, while API tokens and private credentials must be excluded.

For Zenodo, the record DOI and record ID improve repeatability, but records can contain multiple files or versions. The selected file URL and checksum should therefore be

stored. For Git platforms, the clone should record the resolved commit hash rather than relying only on the default branch. These additions would make repeated experiments more comparable than a moving `git pull` against an unspecified revision.

7.8. Interim Assessment

The preliminary evidence supports the engineering feasibility of RQ1 through RQ3: all connectors can produce normalized metadata; the common database can store the results; and the seven relational datasets can be transformed into a linked RDF graph. The evidence is not yet sufficient to compare platform reproducibility rates, metadata quality, or classification accuracy. The final thesis must replace this interim section with results from the controlled and human-labelled samples and provide confidence intervals or appropriately cautious descriptive statistics.

8. Discussion

8.1. Answering the Research Questions

8.1.1. RQ1: A Shared Pipeline with Platform Connectors

The implementation indicates that cross-platform support is most maintainable when acquisition is isolated from execution. GitHub and Codeberg share Git validation and cloning, while Zenodo requires API validation and archive acquisition. Once a local directory and normalized notebook paths exist, the same notebook, requirement, environment, execution, and comparison functions can be reused. This design avoids three copies of the most complex pipeline stages and keeps output semantics comparable.

The result also shows that platform detection alone is insufficient. Identity, batch URL reconstruction, notebook URL normalization, database lookup, metadata retrieval, validation, and semantic typing must all use the platform. A connector abstraction is therefore a cross-cutting contract even when only a few functions are platform-specific.

8.1.2. RQ2: Normalized Metadata

The eight-field contract provides a workable common denominator for descriptive metadata. Title, description, creator, licence, keywords, and timestamps have meaningful counterparts in all source models. DOI is naturally asymmetric: it is a normal Zenodo property and usually absent from Git repositories. Preserving an empty value is more accurate than generating a synthetic DOI or treating the source URL as one.

Normalization necessarily loses some structure. Multiple creators become one semicolon-separated string, licence objects become one identifier or name, and platform-specific metadata not selected by the schema is omitted. This is acceptable for the first relational layer but should be reconsidered for long-term semantic publication. The graph could represent creators and licences as resources when richer queries become necessary.

8.1.3. RQ3: Cross-Platform Provenance

The ontology extension demonstrates how platform provenance and resource type can co-exist. GitHub and Codeberg share `GitRepositoryResource` but link to different platform individuals. Zenodo links to its own platform and uses `ArchivedResearchRecord`. Runs and executions are activities, while repositories and results are entities. This aligns operational pipeline evidence with PROV-O and avoids encoding every stage as a literal column.

The mapping layer is as important as the ontology. An OWL file defines possible classes and properties, but does not populate them. The seven mappings create stable IRIs, types, literals, and relationships from the seven CSV exports. Treating the mapping files as reviewable thesis artifacts makes the relational-to-semantic transformation transparent.

8.1.4. RQ4: Evaluation

Technical tests show that the architecture works for selected examples and that the first graph can be built. They do not yet answer whether metadata quality or reproducibility differs between platforms. The final experiment must control sample selection and report stage-specific denominators. For example, an execution success rate should not silently exclude resources that failed acquisition unless the denominator is clearly defined.

8.2. Design Trade-offs

8.2.1. Shell Orchestration

Extending the Bash baseline minimised disruption and enabled direct reuse of mature processing functions. Shell is effective for composing Git, curl, jq, SQLite, pyenv, and Jupyter commands. Its disadvantages become visible in metadata handling and database safety. Newline-delimited arrays are fragile compared with typed objects, and SQL strings require manual escaping. A future refactoring could retain shell as the user-facing entry point while moving connector and persistence logic into a typed Python module.

8.2.2. Working Database Copy

The copy-on-first-run model protects source evidence and was accepted as an operational compromise. It is simple to explain and supports repeated development. Its main weakness is stale state: once an empty or outdated working database exists, the source is not copied again automatically. A future command should explicitly initialise, refresh, or migrate a working database while requiring confirmation before replacing results.

8.2.3. Current-State Metadata

The metadata upsert keeps one current row per repository. This simplifies display and mapping but removes observation history. APIs can change titles, topics, licences, or timestamps. Longitudinal analysis would benefit from a metadata observation table with source and fetch times. The present schema is suitable for the thesis pipeline but should not be interpreted as a complete metadata provenance archive.

8.2.4. CSV as a Semantic Boundary

Exporting CSV introduces an extra step between SQLite and RDF. Direct database mappings could reduce duplication, but CSV matches the upstream FAIR Jupyter workflow, is easy to inspect, and creates a stable materialisation snapshot. Explicit export queries also allow derived external URLs and resource types to be reviewed independently of RML. For this project, auditability is more valuable than removing one build stage.

8.3. Threats to Validity

8.3.1. Construct Validity

Successful execution is not identical to scientific reproducibility. A notebook can execute without errors and still use different data, stochastic behaviour, or changed services. Con-

versely, a different visual or timestamp output may not invalidate the scientific conclusion. Cell-level output comparison is a useful automated measure but must be interpreted as computational evidence, not a universal judgement about research validity.

Metadata completeness also depends on the chosen schema. Counting a missing DOI against GitHub would unfairly penalise a platform where repositories normally do not have one. Completeness metrics should distinguish universally expected fields from platform-specific fields.

8.3.2. Internal Validity

Network availability, API rate limits, package-index changes, Python build failures, and remote default-branch updates can affect results. The same repository may produce a different outcome on another day without a pipeline code change. Recording commit hashes, file checksums, dates, and environment versions is necessary to reduce this threat.

The current execution environment uses pyenv rather than a fully isolated container per notebook. Although environments separate Python packages, notebooks still share host-level libraries, operating-system tools, and network policy. These factors can influence success and must be held constant during the final experiment.

8.3.3. External Validity

The inherited dataset is dominated by GitHub-hosted biomedical repositories. Newly selected Codeberg and Zenodo resources may come from different communities and project types. Any observed platform difference could therefore be a domain or sampling difference. A balanced sample improves comparison but cannot make all platform populations equivalent. The thesis should describe the sample precisely and avoid generalising beyond it.

8.3.4. Semantic Validity

The first ontology version expresses the intended activity chain, but ontology quality requires more than a successful parser. Domains, ranges, inverse properties, disjointness, and literal datatypes need review. Competency questions should be connected to explicit expected results. External ontology alignment should be checked with domain experts before the model is presented as stable.

8.4. Operational and Ethical Considerations

Executing third-party notebooks can run arbitrary code. A research pipeline should use restricted accounts, resource limits, network controls, and disposable environments. Public deployment would require stronger sandboxing than the development system. The project must also respect API terms, rate limits, software licences, and personal information in repository metadata.

Automated classification introduces additional concerns. Categories may reflect training-data bias, and inferred labels should not be treated as author-provided facts. The graph should distinguish an automated classification activity from a source metadata assertion and record the model version and confidence.

8.5. Integration Outlook

The long-term goal is integration into a Jupyter4NFDI-oriented service. The connector boundary supports adding new platforms without modifying notebook execution. The ontology provides a platform-neutral resource parent and provenance chain suitable for service APIs. Before integration, the implementation needs configuration hardening, safe archive extraction, stronger execution isolation, stable migrations, broader tests, and a documented deployment interface.

The architecture nevertheless provides a useful foundation. It turns platform variation into explicit provenance rather than hiding it, and it connects acquisition decisions with execution evidence. This is important for services that must explain not only whether a notebook failed, but also what resource was processed, where it came from, how it was acquired, which environment was used, and which result was generated.

9. Conclusion and Future Work

This thesis addresses the GitHub bias of an existing Jupyter notebook reproducibility pipeline and its FAIR Jupyter knowledge-graph layer. The implemented system detects GitHub, Codeberg, and Zenodo resources; validates each source with the appropriate mechanism; normalizes descriptive metadata; acquires Git repositories or research archives; and passes all local content through one shared notebook-processing route.

The implementation demonstrates that GitHub and Codeberg can share Git validation and acquisition while retaining separate API adapters. Zenodo requires a genuinely different connector because a record is an archived research object rather than a Git repository. The eight-field metadata contract hides API structure differences from database storage while preserving missing values honestly. A dedicated metadata table and platform-aware repository identity support mixed-platform querying and batch execution.

The database extension is connected to a first semantic-publication workflow. Seven CSV exports are transformed through YARRRML-derived RML mappings, materialised with Morph-KGC, and combined with an ontology extension. The model distinguishes Git repositories from archived records and links platforms, resources, notebooks, repository runs, notebook executions, reproducibility results, and classification activities. Preliminary construction before the classification extension produced 222,048 unique triples and supported initial SPARQL checks.

The current evidence establishes engineering feasibility but not final comparative findings. The hybrid notebook classifier is implemented, including disagreement warnings, human review, evaluation metrics, and semantic export; its labelled evaluation remains unfinished. The next work is to complete the ontology and mapping review, run a controlled cross-platform sample, label a representative notebook sample, and analyse metadata completeness and stage-specific outcomes. The final experiment should record immutable revisions and file checksums, use stronger execution isolation, and report sample-dependent limitations.

Further technical work should replace manual SQL strings with parameterised access, version metadata observations, add archive traversal protection, store Git commit hashes, and expose explicit working-database lifecycle commands. The semantic layer should add more datatype and cardinality tests and evaluate alignment with the complete upstream FAIR Jupyter ontology.

The principal contribution is therefore not only support for two more hosts. It is a platform-aware architecture that preserves the type and provenance of research resources while reusing a common computational reproducibility process. This foundation enables future analysis across repository ecosystems and supports a more interoperable, traceable, and extensible notebook reproducibility service.

A. Technical Appendix

A.1. Main Pipeline Route

The following condensed route can be used to inspect logs or explain the implementation during a demonstration:

1. Check required programs and source all modules.
2. Initialise output directories and the working database.
3. Select single-resource or batch mode.
4. Find or create the repository and create a new run.
5. Detect the platform and normalize supplied notebook paths.
6. Validate through Git or the Zenodo Records API.
7. Fetch and store normalized metadata.
8. Clone, pull, or download and extract the resource.
9. Register supplied notebooks or discover `.ipynb` files.
10. Check total and Python notebook counts.
11. Build requirements and create a pyenv environment.
12. Execute notebooks and classify errors.
13. Compare original and executed outputs.
14. Clean the environment and finalise the run.

A.2. Representative Database Queries

Listing A.1: Inspect normalized metadata by platform.

```
SELECT r.platform,
       r.repository,
       m.title,
       m.authors,
       m.license,
       m.doi
FROM repositories AS r
LEFT JOIN repository_metadata AS m
  ON m.repo_id = r.id
ORDER BY r.platform, r.repository;
```

Listing A.2: Inspect repository run outcomes.

```

SELECT r.platform,
       rr.run_status,
       COUNT(*) AS runs
FROM repository_runs AS rr
JOIN repositories AS r
  ON r.id = rr.repository_id
GROUP BY r.platform, rr.run_status
ORDER BY r.platform, rr.run_status;

```

A.3. Representative SPARQL Queries

Listing A.3: List resources and platforms.

```

PREFIX nf: <https://w3id.org/notebookfair#>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>

SELECT ?resource ?resourceLabel ?platformLabel
WHERE {
  ?resource a nf:RepositoryResource ;
            rdfs:label ?resourceLabel ;
            nf:hostedOnPlatform ?platform .
  ?platform rdfs:label ?platformLabel .
}
ORDER BY ?platformLabel ?resourceLabel

```

Listing A.4: Constraint check for Zenodo typing.

```

PREFIX nf: <https://w3id.org/notebookfair#>

SELECT ?resource
WHERE {
  ?resource nf:hostedOnPlatform
    <https://w3id.org/notebookfair/platform/zenodo> ;
    a nf:GitRepositoryResource .
}
# Expected result: zero rows.

```

Listing A.5: Trace an execution to a result.

```

PREFIX nf: <https://w3id.org/notebookfair#>

SELECT ?repository ?run ?execution ?result ?score
WHERE {
  ?repository nf:hasRun ?run .
  ?run nf:hasExecution ?execution .
  ?execution nf:hasReproducibilityResult ?result .
  ?result nf:reproducibilityScore ?score .
}

```

A.4. Build Commands

Listing A.6: Metadata test in the local container image.

```
docker build -f binder/Dockerfile -t notebookfair-test .  
docker run --rm notebookfair-test bash tests/test_metadata.sh
```

Listing A.7: Build the pipeline knowledge graph.

```
python code/fairjupyter/build_pipeline_kg.py
```

The graph builder writes the combined graph and build report to:

```
output/kg/notebookfair-pipeline.nt  
output/kg/build-summary.json
```

A.5. Draft Completion Checklist

- [] Review final title and student administration details.
- [] Complete ontology domain, range, inverse, and datatype review.
- [] Freeze and run the cross-platform experiment sample.
- [] Add comparative tables and plots with denominators.
- [x] Implement hybrid notebook classification and human-review warnings.
- [] Evaluate notebook classification against a human-labelled sample.
- [] Replace interim evaluation statements with final results.
- [] Review references, language, and university formatting requirements.
- [] Obtain supervisor feedback and complete the declaration page.

Bibliography

- [1] Arenas-Guerrero, J., Chaves-Fraga, D., Toledo, J., Perez, M.S., Corcho, O.: Morph-kgc: Scalable knowledge graph materialization with mapping partitions. *Semantic Web* 15(1), 1–20 (2024)
- [2] CERN: Zenodo rest api documentation. Online documentation (2026), <https://developers.zenodo.org/>, accessed 18 August 2026
- [3] Codeberg e.V.: Codeberg and forgejo api documentation. Online documentation (2026), <https://codeberg.org/api/swagger>, accessed 18 August 2026
- [4] Cyganiak, R., Wood, D., Lanthaler, M.: Rdf 1.1 concepts and abstract syntax. W3c recommendation, World Wide Web Consortium (2014), <https://www.w3.org/TR/rdf11-concepts/>
- [5] GitHub: Rest api endpoints for repositories. Online documentation (2026), <https://docs.github.com/en/rest/repos/repos>, accessed 18 August 2026
- [6] Harris, S., Seaborne, A.: Sparql 1.1 query language. W3c recommendation, World Wide Web Consortium (2013), <https://www.w3.org/TR/sparql11-query/>
- [7] Heyvaert, P., Dimou, A., Meester, B.D., Verborgh, R.: Yarrml: A human readable text-based representation for declarative linked data generation rules. In: *Companion Proceedings of The Web Conference 2018*. pp. 1–4 (2018)
- [8] Lebo, T., Sahoo, S., McGuinness, D.: Prov-o: The prov ontology. W3c recommendation, World Wide Web Consortium (2013), <https://www.w3.org/TR/prov-o/>
- [9] Morph-KGC contributors: Morph-kgc documentation. Online documentation (2026), <https://morph-kgc.readthedocs.io/>, accessed 18 August 2026
- [10] Pimentel, J.F., Murta, L., Braganholo, V., Freire, J.: A large-scale study about quality and reproducibility of jupyter notebooks. In: *Proceedings of the 16th International Conference on Mining Software Repositories*. pp. 507–517 (2019)
- [11] Project Jupyter: nbconvert documentation. Online documentation (2026), <https://nbconvert.readthedocs.io/en/latest/>, accessed 18 August 2026
- [12] pyenv contributors: pyenv: Simple python version management. Software repository and documentation (2026), <https://github.com/pyenv/pyenv>, accessed 18 August 2026
- [13] Samuel, S., Mietchen, D.: Computational reproducibility of jupyter notebooks from biomedical publications. *GigaScience* 13, giad113 (2024)

- [14] Samuel, S., Mietchen, D.: Fair jupyter: A knowledge graph approach to semantic sharing and granular exploration of a computational notebook reproducibility dataset. *Transactions on Graph Data and Knowledge* 2(2), 4:1–4:24 (2024)
- [15] Wilkinson, M.D., et al.: The fair guiding principles for scientific data management and stewardship. *Scientific Data* 3, 160018 (2016)